Specialised AI Assistants for the Interpretation of Biopharmaceutical Data

Samuel Stride supervised by Dr Rui Carvalho

***Abstract*—This paper investigated the use of LLM based multi-agent systems (MAS) to automate data analysis and explore the possibility of Edge AI and the self-driving lab. A MAS design was proposed to improve the reasoning ability of small LLMs due to previous findings of the advantages of agent-based exploration and human-inspired peer review. A MAS was designed to automatically process tabular data of any size for application within the biopharmaceutical domain. A hierarchical system, with distinct planning and coding phases, was provided with configurable episodic memory to assess the use of exploration-based learning. Ablation studies were performed to adjust episodic memory placement, self-refinement iteration number and LLM size. Results indicated that memory in the coding phase improves the quality of LLM-judged reports, but at the possible expense of higher debugging effort. Isolated planning memory requires more iterations but at a possible lower debugging cost, producing the highest quality output according to LLM-based evaluations and software metrics, such as code execution success rate and cyclomatic complexity. Whilst more evidence is required to deliver conclusive results, early findings support a narrowing of the performance gap (scaling law) between small 4B (billion) and larger 27B parameter models. These early signs of practical Edge AI may provide industry with a route to overcoming current cost and security limitations of cloud AI.**

***Index Terms*—Large Language Model (LLM), Multi-Agent System (MAS), Retrieval Augmented Generation (RAG), Episodic Memory, Edge AI.**

I. Introduction

THE biopharmaceutical sector is one of an increasing number that seek to benefit from Industry 4.0: often described as the era of smart manufacturing leveraging automation and optimisation to increase productivity and efficiency [1]. Process analytical technology (PAT) has been promoted by regulatory bodies for the improvement of monitoring and controlling biopharmaceutical processes [2]. Significant volumes of multivariate data are increasingly common within these processes, resulting in a shift towards automation such as Machine Learning (ML). However, there are notable financial risks to incorrect algorithms, often requiring extensive domain-specific expertise to effectively utilise [2]. Artificial Intelligence (AI) is a novel technology that is being used to further improve automation as it is becoming increasingly capable. For newer processes, which may lack the multitudes of data required for typical ML, AI may be able to act as a proxy for in-house expertise.

Recently, Large Language Models (LLMs), a type of AI for natural language processing (NLP), have been incorporated into collaborative systems between role-specific instances called agents [3]. Such multi-agent systems (MAS) may expand upon standalone LLM reasoning abilities to decompose complex tasks and perform operations such as data analysis. This is an emerging trend that seeks to realise the concept of the self-driving lab (SDL) [4].

Despite the apparent advantages of AI, data privacy risks limit industry uptake. Cloud-based AI requires the transmission of all information over third-party services [5], presenting a GDPR and confidentiality challenge. The remaining option, though costly, is local infrastructure — a high-risk, high-reward approach.

The assumption that larger LLM models offer greater reasoning ability [6] has driven the financial risk of local AI. LLM weights and KV-cache (used to incrementally store calculation results) are responsible for requiring expensive state-of-the-art hardware [7]. For reference, OPT-175B (a 175 billion parameter model) may use 325 GB for the model weights and 950 GB for KV-cache, requiring 16 NVIDIA A100 80 GB GPUs [7], each with an estimated price of £11,000 - £12,500 [8]. This cost burden far-exceeds the theoretical advantage of LLM-driven automation for many organisations, inhibiting the progress towards the self-driving lab.

Due to this limitation, Edge AI is increasing in demand due to the minimal setup costs. Edge computing, which typically refers to performing computations nearer to a data source [9], points to local infrastructure to replace the need for data centres and cloud-AI. However, memory constraints of edge devices, such as phones, laptops and embedded systems remain problematic. Various optimisation, compression and architectural design strategies seek to alleviate this problem and enable smaller LLMs to achieve comparable reasoning to larger models — a direct contrast of the scaling law [10], [6]. Recent advances in increased reasoning of smaller LLM models such as the Qwen models and Google DeepMind Gemma 4 support this direction but are only one part of the solution towards enabling Edge AI [11], [12].

Individual reasoning ability is often enhanced by a formal peer-review mechanism, such as that employed in academia [13]. Analogous to human collaboration, AI generated outputs can benefit from concept exploration and peer-review-style self-refinement because definitive statements of correctness (ground truth) are elusive to both humans and AI alike [14]. Post-hoc correction [15] can be applied to AI through the design of a MAS, which has been shown to improve reasoning outcome [14]. Whilst the scaling law (larger LLM equals higher quality) remains central [10] to current observation, MAS design may hold the key to weakening its influence.

Motivated by this concept, this project aims to deliver an ablation study for an LLM-based multi-agent system to assess MAS design decisions and study the implications of

model size on data analysis task performance. This study seeks to answer the following research questions: **1. Can MAS design choices mitigate the limitations of the scaling law? 2. Does introducing memory at different stages of a MAS improve the effectiveness of self-refinement? 3. Does iteration number influence the effectiveness of self-refinement?**

II. Literature Review

A. Large Language Models (LLMs)

True automation is underpinned by a capacity to adapt and evolve, akin to the human thought process. Whilst rule-based systems are inherently inflexible, artificial intelligence (AI) which is based on bio-mimicry, marks a crucial advancement towards the self-driving lab [4]. Cognitive processes in humans can rely on internal dialogue, mental imagery and intuition amongst other mechanisms that result in reasoning ability. Large language models (LLMs) were created to exhibit a form of reasoning through natural language but the ability of an LLM to automate reasoning is afflicted by errors (hallucination), a consequence of creativity in inference-based systems [16]. This issue was anticipated by Turing who suggested that machines, analogous to a child's mind, would rely on learning [17], thus LLMs are trained on a large corpora of information. Brown et al. [6] noticed that the ability to rely on such quantities of pre-trained knowledge enabled in-context learning — the defining feature enabling adaptation. A scaling law emerged, indicating that larger models were shown to produce higher quality outputs, but practical challenges with input size (context-window) persisted. Multiple LLM instances (agents) combat this limitation by distributing reasoning across many specialist roles, as enabled by the Autogen (now AG2) framework [18]. A method called ReAct [19] drove LLM-powered automation by encouraging externalisation of reasoning, which subsequently triggers functions to enable software environment interaction. Thus, the challenge of building autonomous machines transitioned into the orchestration of multiple agents.

B. LLM based Multi-Agent Systems

The finite context-window limits the holistic understanding that is achieved by the provision of a complete fact-base, which may otherwise have reduced hallucination — the concept of LLMs confidently asserting falsehood [20]. Injecting (relevant but concise) chunks of information into the input prompt to accompany a task description is the basis for the variety of retrieval augmented generation (RAG) mechanisms. A commonly used retrieval method is the similarity search [20]. Documents are divided into chunks, which are converted into vectors, or embeddings, that encode semantic meaning. An agent can query this information store, or vector database, to retrieve the embeddings that represent the highest semantic similarity to the question — often using k-nearest-neighbours. Alternatively, another approach is to use a GraphRAG [21]. This uses pre-determined knowledge graphs (KGs) to link concepts (nodes) based on their relationship (edges) regardless of semantic similarity. This method may provide more comprehension in domain-specific tasks, but it requires significant time and domain expertise to construct, which is often impractical. Other ideas for retrieval explore map-reduce strategies [20] that rely on repeated summarisation of sections of large documents until a single manageable summary emerges. However, there is the risk that important information is overlooked.

To achieve autonomy, multi-agent systems must have both appropriate architecture and agent-to-agent interaction to solve complex problems. Zhao et al. [22] summarised the key important architectures; centralised, decentralised and hierarchical structures. Centralised systems have a central agent that is responsible for task decomposition and delegation. Whilst being a highly flexible system architecture, the central agent becoming overwhelmed is a typical failure mode. Decentralised systems rely on peer-to-peer conversational styles, where each agent processes the current state of the task, to respond and collaborate with other agents accordingly. This may allow individual agents more flexibility, as well as shifting the failure modes away from any individual agent. However, this lacks the governing structure that is required for data analysis tasks. Hierarchical architectures rely on a tree-structure, where agents often perform actions at different levels of abstraction. Initially, there may be a planning phase at high levels of abstraction, allowing for task decomposition, whilst lower levels may need to act on a plan in a more technical manner. Despite its abilities in task decomposition, this architecture can constrain creativity due to its rigid structure. Within any chosen architecture, which is selected based on the task, agent-to-agent interaction style is a critical design choice. Agents can cooperate by influencing the reasoning steps towards common goals; they can compete against each other with opposing goals to uncover reasoning flaws; or they can negotiate, using a hybrid approach, which is common in planner-reviewer interactions. As the cost of AI flexibility and creativity is inference, which can lead to incorrect lines of thinking, a combination of supportive comments and criticism is aligned with the scientific peer review [13], indicating that data analysis tasks may benefit from a negotiation-style.

This idea of LLM collaboration through negotiation is typically achieved through iteration loops, using a technique called self-refinement, to enable convergence of the output quality [15], [14]. These loops may be repeated N times, requiring agents to re-write their outputs based on a selected evaluation strategy. Wang et al. [23] explored the applications of using LLM-based evaluations by comparing the feedback and scoring delivered by LLMs to that of a human expert fulfilling the same role. They found that larger models (>100 billion parameters) exhibited more human-like judgments, further supporting the presence of scaling law limitations. It was also found that this effectiveness was task-dependent, highlighting the need for ground truth validation for most practical LLM-based evaluation to be trusted. However, the benefit is the reduced requirement for human expertise, which would be required for a human-in-the-loop (HITL) style evaluation, which contradicts the specification of a fully autonomous system. Triem and Ding [24] suggested that human-based opinions are more strongly weighted than other agents and

can lead to improved performance in evaluation capability but this would rely on in-house expertise. Another evaluation strategy is to use absolute verification of LLM outputs, the automation of which is typically exclusive to coding based tasks, as the executor or compiler provides clear success or failure signals. This is supported by the work of Li et al. [25] which utilised execution-based evaluation as an initial indicator of correctness. Building on the work of self-refinement, Shinn et al. [26] introduced Reflexion, which used episodic memory, allowing agents to access a history of past mistakes before providing an alternative output. This was shown to drastically improve performance in comparison to simple trial and error approaches that ReAct [19] methods offer. However, the selection of the evaluation strategy impacted the observed improvements, indicating that memory must be appropriately designed within a system. Reflexion is a cheaper alternative to updating LLM model weights and so it is easy to incorporate into many MAS designs. Data analysis often requires trade-offs and intelligent design decisions due to domain constraints, hence self-reflection, regardless of episodic memory, is critical to the adaptability of AI [14].

AI and MAS are rapidly developing fields that have been used in applications exceeding simple chatbots to enable the progression towards true automation. A self-driving codebase was the subject of a project conducted by a team at Cursor [27]. Lin discussed the design considerations that enabled a system to run continuously for a week to develop a web browser from scratch without human intervention. Many system iterations were discussed with different agent coordination styles. One method involved agents running in parallel, using a coordination file to lock a task until an agent branch was finished. Whilst another approach used a centralised architecture with a main planner/executor agent delegating to all parallel workers. They finally settled on a hierarchical system of a main planner with sub-planners and workers, which operate with high parallelism on a Git repository. The limitation of the centralised architecture was found to be a scaling problem, causing the central-agent to become overwhelmed and fail to fulfill its role effectively. Among all iterations, a clear theme of judging and self-refinement was employed, often stating that accepting mistakes but relying on the self-correcting nature of a converging system was necessary: a confirmation of AI limitations and the adaptability–correctness trade-off. It was clear that the use of Git was a natural fit with coding tasks and may have contributed to the designed stateful history that enabled convergence by learning from past attempts. Experimentation was held in higher regard than excessive rules, indicating a learning mechanism that may be suitable for other domains, such as data analysis. Despite inherent limitations of the centralised architecture, CMBAgent [28] — a cosmological data analysis MAS — successfully used this for smaller-scale tasks, reporting data analysis outcomes that were very similar to that of a PhD researcher. CMBAgent placed more emphasis on an initial planning phase, as opposed to dynamic planning, indicating better alignment with data analytics compared with pure software engineering tasks. Data analysis was proven to be within the skill set of LLMs as they can propose code based on rigorous plans. CMBAgent championed the use of a RAG system to enhance factual accuracy and enable the use of specialist python packages that are not guaranteed to have been part of the LLM training dataset, effectively demonstrating in-context learning. In extension to the concept of LLM-based MAS for data analysis applications, the AI Scientist [29] produced by Lu et al., operationalised end-to-end automation that is the crux of the self-driving lab [4]. This work exceeded a simple execution stage and sought to enable parallel execution of ideas, with mechanisms for parameter optimisation, to select the most appropriate methods before an ablation study stage, thus more closely mimicking the academic data analysis process. It is clear that the enhancement of the execution phase, suggested in CMBAgent [28], is another application of agent collaboration through a self-refinement approach. The AI Scientist was capable of its own peer review via LLM evaluation and additionally used multimodal AI to provide another dimension for review, which is becoming an increasingly popular trend [29], [30], [31]. It is clear that LLM evaluations only become meaningful when compared with human judgment. Most notably, this work delivered research papers, of which one passed the acceptance criteria for the ICLR workshop [29]. Reflecting on Turing's Imitation Game, the AI Scientist is a compelling demonstration of the progress towards machine responses being indistinguishable from that of a human [17]. Even so, it is clear that the limitations of AI are due to hallucination which has proven to be more challenging in open-ended research problems. The progression of these systems proposes a convincing argument for the use of MAS in automating data analysis systems.

III. Methodology

Aligned with the efforts towards the self-driving lab [4], an LLM-based multi-agent system has been designed for the analysis and interpretation of high performance liquid chromatography (HPLC) data in the biopharmaceutical domain, as shown in Figure 1. Despite this domain focus, it should be noted that the designed system is non-specific towards the domain of the data. A hierarchical system was chosen due to the limitations of other architectures for data analysis tasks [27], [29]. Self-refinement was used to mimic the human peer-review process and further enhance reasoning capability [15]. Intermediate LLM outputs, marked with green file icons, are added to a shared state, with relevant selections hand-picked depending on the agent role. The creativity level [32], determined by LLM temperature, was set to 0.2-0.3 to encourage factual accuracy with some exploration allowed. The system runs within a single container (SingularityCE) for ease of deployment and security considerations. To mitigate the risk of running AI generated code, the agents are provided with a scratchpad to write to, whilst all other files are given read-only permission.

A. Data Discovery, Research and Focus Area Phase

Data analysis requires a deep comprehension of a dataset, which humans obtain by visually scanning information and conducting research, whilst LLMs are limited by the context window. In this system, tabular data is required as an input,

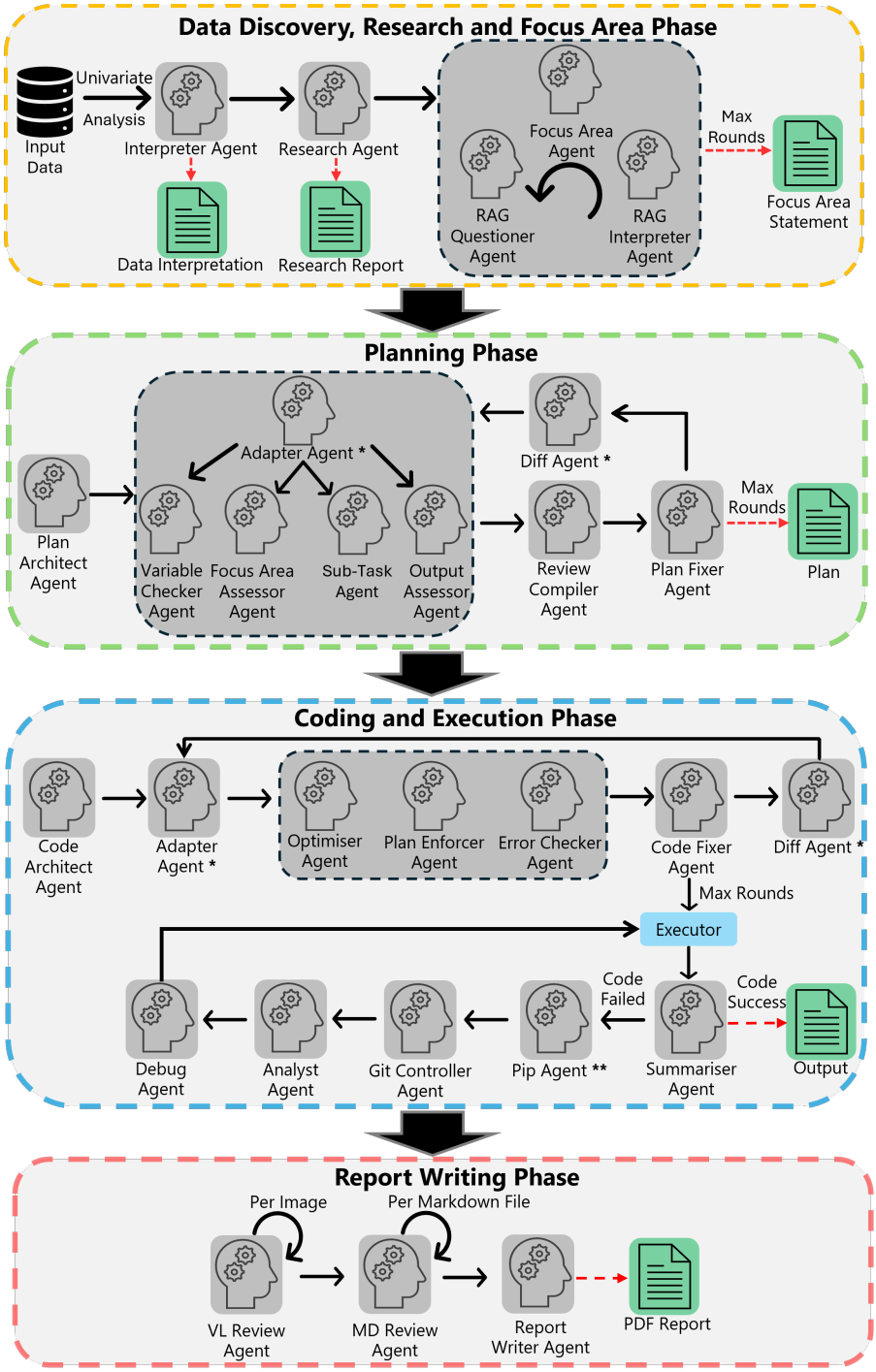


Fig. 1: Multi-Agent System Design

which is processed by a deterministic function that extracts categorical metadata and uses univariate statistics to describe the dataset. This enables the interpreter agent to create a detailed but concise report. A research agent, powered by the AG2 deep research MAS [33], [34], is then able to create a knowledge base that is RAG-accessible. A focus area agent then seeks to define a brief for what constitutes data analysis within the domain by iteratively refining a focus area statement. Refinement is achieved by collaboration between a RAG questioner — for similarity search retrieval, using the all-MiniLM-L6-v2 sentence embeddings model [35] — and an interpreter that provides feedback. Hence, this phase seeks to provide an alternative method to human processes and facilitate effective planning from inferred dataset understanding.

B. Planning Phase

Planning in MAS, as enabled by a hierarchical system, has been shown to be critical within a scientific domain as methods and parameters must be carefully selected to suit the task [29], [27], [28]. This process begins with a plan architect,

that uses the inferred dataset understanding, to draft an initial plan with a maximum number of allowed plan steps specified. Structured outputs (template filling) were found to improve plan coherence during this phase and enables the subsequent panel of specialist reviewers to more effectively assess positive and negative aspects in a negotiation style [22] to ensure a more well-rounded criticism. Four distinct reviewer roles were implemented to assess several dimensions of the plan, but as each dimension is not necessarily orthogonal, a compiler agent was employed to resolve conflicts according to knowledge of each reviewer's specialism. A plan fixer was designed to be an instruction follower in contrast to the plan architect, which acts independently, thus completing the self-refinement loop [15], [14]. Agents marked with an (*) in Figure 1 contribute towards episodic memory, which is achieved by using a Git repository [27], to enable the review panel to learn from past exploration and mistake. A Git repository is created for each review specialism to restrict access to relevant roles. The adapter agent is responsible for maintaining a record of issues and the corresponding resolution state for each review specialism. The adapter uses a scoring signal to identify whether the plan is improving or degrading relative to each review specialism. Additionally, this agent seeks to identify undesirable review patterns to encourage convergence [27]. A diff agent uses in-built Git diffs between plan versions to provide reviewers with an independent report of changes made, to prevent reliance on re-discovery of issues in each iteration.

C. Coding and Execution Phase

As LLMs rely on natural language, coding is an effective mechanism to support data analysis implementation [28]. Whilst effective planning is important to successful code, smaller errors can remain, which increases computational cost to debug when relying on execution alone. As such, a preemptive code refinement stage is implemented as a carbon copy of the planning refinement loop, with specialist code reviewers and identical options for episodic memory. This stage is repeated for each independently executable step in the plan to ensure undivided agent attention. Once the code has been drafted and refined, it is passed to an executor. If execution succeeds, the code output is collected and the system restarts for the next plan step. If unsuccessful, the summariser agent concisely explains the issue and has the ability to dynamically enable the pip installation agent, marked by (**), if missing python packages are found. The Git controller agent submits the execution status and code to a Git repository and can choose repository rollback if the latest code deviates from the plan or if the history suggests any regressions. The analyst uses the past history, the current code and the execution error to instruct the debug agent to fix the code. This is reminiscent of Cursor's approach to allowing small errors with self-correction based on memory [27].

D. Report Writing Phase

Explainability is central to allowing LLM output to be useful regardless of correctness. Whilst small mistakes — which are the tradeoff for adaptability [17] — can propagate through a MAS and limit overall correctness, work done can provide a foundation on which experts can build, to increase productivity. The report writing phase leverages multimodal models, such as visual language (VL) models, for assessing both graphical output [29] and text based analysis. The combination of these generated insights, the dataset interpretation and the intended analysis plan enables a report writer to deliver on explainability criteria to provide a human-readable pdf, using a LaTeXcompiler. All code files are added to an appendix to enable reproducibility of results and a stronger starting point for human expertise.

TABLE I: Evaluation Metrics

Category	Metric	Explanation
Coding Metric	Cyclomatic Complexity	Number of decision points (such as if-statements, for loops, etc) in the code
Outcome Metrics	Success Rate	Proportion of successful code executions
	Mean Debug Attempts	Average debug attempts until successful code execution
	LLM-as-a-Judge	Assessment of the final report quality

E. Experimental Setup and Evaluation

To deliver an answer to the main research questions, an ablation study was conducted on the MAS in Figure 1. To assess the relevance of the scaling law (**RQ1**), the system was run using Qwen3.5-4B, a model small enough to enable Edge AI, and a larger model to act as a baseline result, such as Qwen3.5-27B, which requires expensive hardware. For each LLM size, an ablation study enabled four distinct combinations of episodic memory across the planning and coding stages to assess the effect of memory placement on self-refinement effectiveness (**RQ2**). Finally, the 4B parameter model was subject to further testing by adjusting the number of self-refinement loops (from 5 to 10), within both planning and code-drafting phases, across the four memory configurations (**RQ3**).

To evaluate these research questions, 20 pre-defined tasks were used to assess the system across all design configurations. These tasks covered a range of queries that focused on both analysing the data and assessing the quality of the data and experimental conditions. A maximum of three plan steps was used, enabling each run to obtain up to 60 observations. The 4B parameter model frequently experienced response-generation timeouts that could not be resolved within an acceptable number of retries. To address this, the runs were repeated with the same set of tasks to ensure a sufficient number of observations were recorded for interpretation. Due to computation time constraints, only the data for 5 iterations in the planning and coding stages produced enough observations (80-120) to enable bias-corrected and accelerated (BCa) bootstrapping (with replacement) to obtain 95% confidence intervals for one million resamples. Hence, this study presents early findings towards answering the research questions.

The evaluation metrics for this system are shown in Table I. Selecting an evaluation strategy proved to be difficult owing to the lack of human-generated comparisons to validate the system output against human-observed ground truth. As such, these evaluation metrics attempt to study the output quality of both the code and the final report as these are both crucial to data analysis task performance and the value of the final interpretations. The cyclomatic complexity measures the number of decision paths possible within any given code run, such as if-statements, in addition to other code statement types, [36], [37]. A higher cyclomatic complexity is typically associated with a higher probability of coding errors, with 10 being a typical upper practical limit for humans. The capability of a MAS to either avoid complexity or reliably use complex code may provide insights into the reasoning ability for a given design choice. Thus, the complexity can also act as an explanatory variable for the outcome metrics shown in Table I. The success rate is the proportion of individual coding steps where the execution succeeded within an allowed maximum number of debugging attempts, chosen in this paper to be 20. To enable the self-driving lab, exploration of different approaches may be crucial, hence it is important that a MAS is capable of successfully implementing different ideas efficiently. Thus, an effective data analysis MAS must have a high success rate to reduce latency, that limits current human endeavour, to enable the benefits of automation. Further to this, the average number of debugging attempts is suggested, as this is a measure of the amount of self-corrective action that is required within this MAS. Depending on the domain of research, computational cost can be a limitation of both funding and available time. Thus, a system that has a high average debugging effort required may not be suitable for many domains that would otherwise benefit from automation. These are direct measures of the coding phase effectiveness and they may act as indirect measures of the effectiveness of the planning phase. Thus, in an ablation study, which isolates changes, these metrics may be used to infer the effect of episodic memory and LLM size on reasoning quality. The LLM-as-a-Judge metric can be a proxy for assessing the scientific rigour of the data analysis process [29], [23]. However, this paper does not provide a comparison to human reasoning ability and so cannot make claims relative to human-established ground truth. Despite this, relative scoring between design configurations is useful as an early prediction of the effects of MAS design and LLM size on the scientific rigour and efficacy of data analysis, enabling a meaningful contribution to answering the research questions. LLM-as-a-Judge has been implemented by using the Anthropic Claude-Haiku-4.5 [38] model to analyse pdf documents and provide a score based on a provided rubric. Although this uses cloud-AI, it was determined that data security is preserved. Despite providing interpretations and data analysis results, the dataset is never uploaded over the cloud. The results of the judging scores were provided in distinct categories, with a power mean (p=-2) used to aggregate scores into an average. This was done to penalise the lowest scoring results to prevent any significant weakness from being overlooked in the final scoring, thus presenting a non-compensatory metric [39].

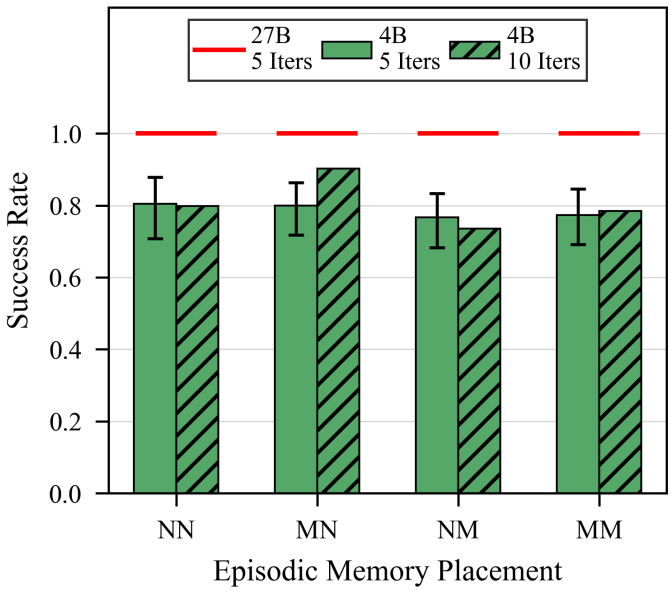


Fig. 2: **Success rate across design configurations for Qwen3.5-4B.** Episodic memory is shown as N (No Memory) and M (Memory). The order is XY, where X is planning phase configuration and Y is coding phase configuration. The data is shown for 5 and 10 iterations in the planning and coding phases, respectively. 95% confidence intervals from a BCa bootstrap method are shown for 5 iterations only. The baseline for Qwen3.5-27B using 5 is shown in red for each ablation.

IV. Results and Discussion

The results for the ablation study, shown in figures 2-6, are presented to address the main research questions: **1. Can MAS design choices mitigate the limitations of the scaling law? 2. Does introducing memory at different stages of a MAS improve the effectiveness of self-refinement? 3. Does iteration number influence the effectiveness of self-refinement?**

A. Effects of Episodic Memory Placement in MAS Design on Performance and the Scaling Law

Despite recent advances in the capabilities of smaller LLMs, the results are consistent with prior findings of the effects of the scaling law [10]. However, MAS design choices, such as the placement of episodic memory, appear to narrow part of the observed performance gap between model sizes as agents learn from past mistakes [26]. To support a direct comparison, this section focuses on the design configuration with five iterations in both the planning and coding phases. Figure 2 shows the impact of the scaling law through the success rate metric. The 27B parameter model shows a 100% success rate regardless of memory configuration. By contrast, the smaller LLM (4B parameters) has success rates ranging between 78-80% with the coding memory being associated with a small reduction. The confidence intervals at the 95% level shown for the 4B parameter model indicate reasonable precision in the estimated success rate owing to the 16% interval width. The large degree of overlap indicates that memory placement has minimal effect on the success rate.

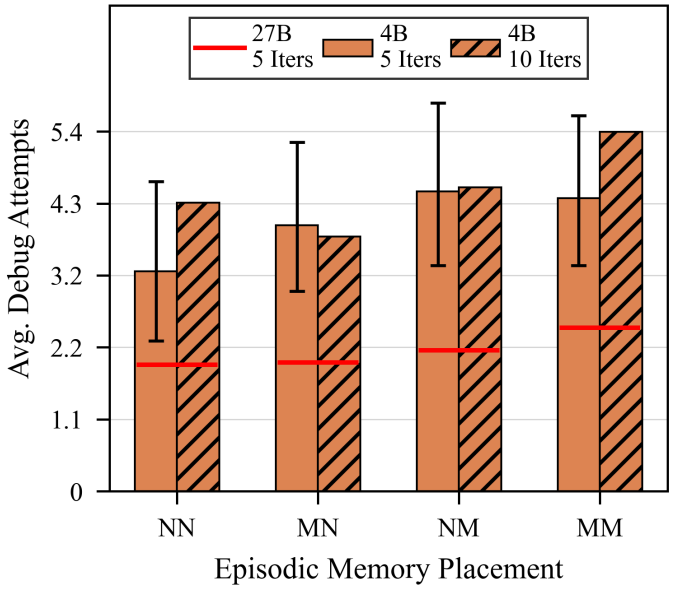


Fig. 3: **Average Debug Attempts across design configurations for Qwen3.5-4B.** Episodic memory is shown as N (No Memory) and M (Memory). The order is XY, where X is planning phase configuration and Y is coding phase configuration. The data is shown for 5 and 10 iterations in the planning and coding phases, respectively. 95% confidence intervals from a BCa bootstrap method are shown for 5 iterations only. The baseline for Qwen3.5-27B using 5 is shown in red for each design configuration.

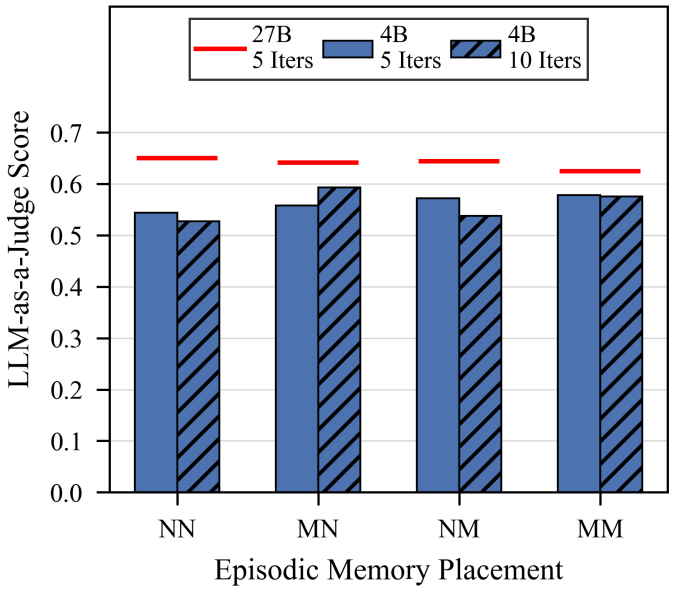


Fig. 4: **Average LLM-as-a-Judge scores across design configurations for Qwen3.5-4B.** Episodic memory is shown as N (No Memory) and M (Memory). The order is XY, where X is planning phase configuration and Y is coding phase configuration. The iterations are (5-5) and (10-10) for the number of iterations in planning and coding phases, respectively. The baseline for Qwen3.5-27B using 5 is shown in red for each design configuration.

In contrast, the episodic memory appears to influence the average debugging attempts, as shown in Figure 3. This effect is most noticeable for the 4B parameter model, which sees an increase in the mean from 3.5-3.8 to 4.4-4.6 due to coding memory. However, the confidence intervals (at 95%) suggest low precision due to the width and overlap between memory configurations. This suggests that more data would be required to report a definitive effect. Whilst for most use cases an extra debugging attempt is unlikely to be problematic, an increased latency per task may become a cumulative issue both for productivity and cost. The 27B parameter model shows more execution robustness as the average debugging attempts are localised to the 2-2.5 range. This is consistent with the perceived scaling law effects in this study.

To understand the observed trends in the average debugging attempts, the cyclomatic complexity was measured. The cyclomatic complexity is the number of decision paths that exist within code with a higher complexity representing more decision logic implementation [37]. A higher code complexity can often result in code that is more difficult to understand and debug, with a cyclomatic complexity of 10 being a typical maximum advisable threshold for humans [36], [37]. Figure 5 shows the complexity for the 4B parameter model, which appears to increase from a mean of 9 up to 12-13 (above the recommended threshold) with the introduction of episodic memory on the coding stage. This may be responsible for the observed increase in the average debugging effort required for the same memory configuration. Coding memory seems to exhibit a larger spread in complexity compared with no coding memory, as the peak complexity was observed to increase to 47. It is not clear whether this memory location is more sensitive to task difficulty, but planning memory appears to be the most consistent as the distribution of complexity is centered around lower values.

The complexity scores for the 27B parameter model seem to exhibit a similar trend to the smaller LLM, as shown by Figure 6. However, the mean complexity is several times higher for the larger model with means ranging from 22-30, exceeding the recommended limit. It could be argued that the minimal impact of the complexity on both success rate and average debugging attempts is further evidence of the previously determined stronger reasoning abilities of larger LLMs [10].

Whilst the effects of a higher complexity appear to be undesirable, particularly for smaller LLM sizes, the effect on the output quality can only be determined by assessing the quality of the end report. The LLM-as-a-Judge scores, as shown in Figure 4, may act as a rough approximation to a human evaluator [29], [23]. The 4B parameter model scores appear to improve from a minimum of 54% to a maximum of 58% with the introduction of both planning and coding memory. The results indicate that coding memory may lead to a higher output quality, and it is possible that a higher coding complexity may be a contributing factor. This idea is further supported by the 27B parameter model which attains a near-constant 65% score across all configurations of episodic memory whilst producing code that has a high degree of complexity. However, further study would

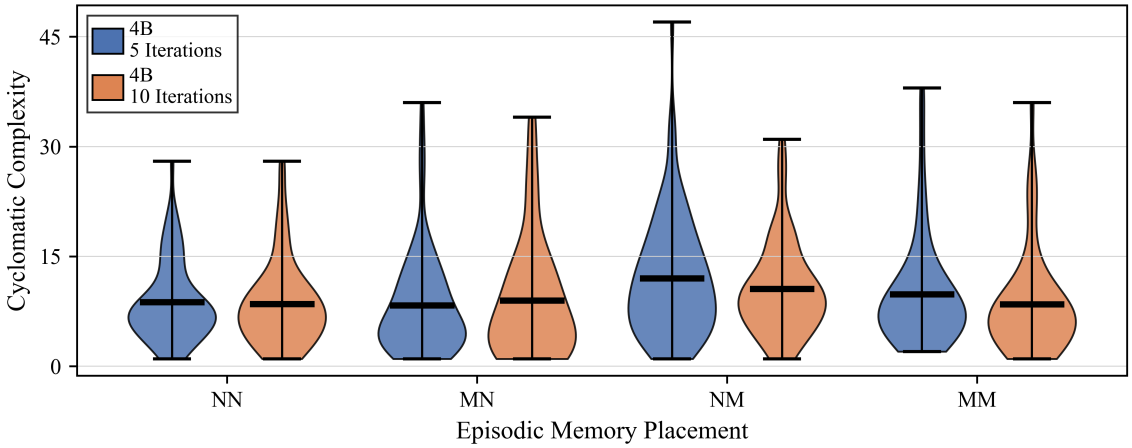


Fig. 5: **Violin plot of cyclomatic complexity across design configurations for Qwen3.5-4B.** Episodic memory is shown as N (No Memory) and M (Memory). The order is XY, where X is planning phase configuration and Y is coding phase configuration. The iteration numbers are 5, shown in blue, and 10, shown in orange, for both planning and coding phases.

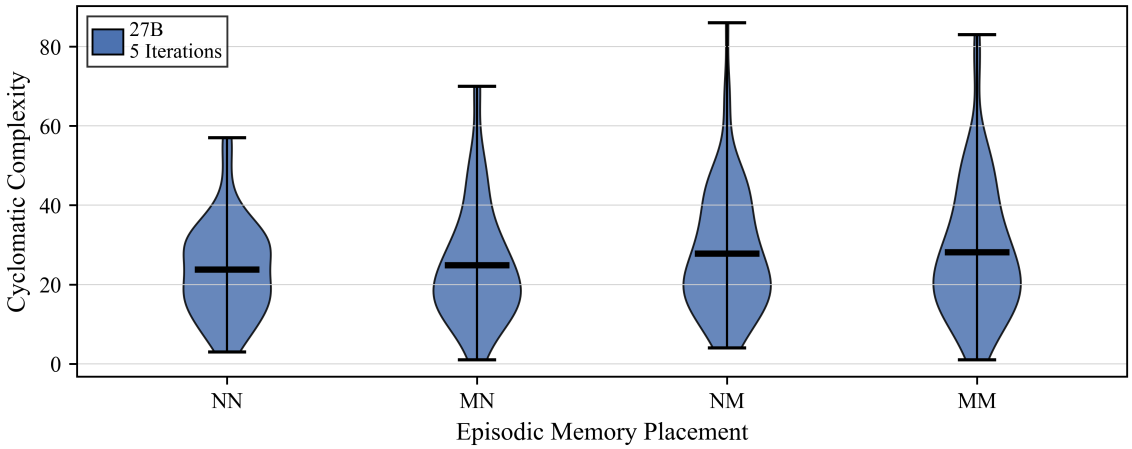


Fig. 6: **Violin plot of cyclomatic complexity across ablations for Qwen3.5-27B.** Episodic memory is shown as N (No Memory) and M (Memory). The order is XY, where X is planning phase configuration and Y is coding phase configuration. 5 iterations were used on the planning and coding phases, respectively.

be required to deliver conclusive evidence. Nevertheless, it appears that strategic memory placement and MAS design — namely self-refinement — may contribute to narrowing the observed performance gap predicted by the LLM scaling law [10] by enabling agents to learn from past mistakes [26]. Memory appears to be most beneficial for smaller LLMs with coding memory leading to an improved report quality despite a tradeoff with debugging effort.

B. Effect of Iteration Number on Self-Refinement Performance

The concept of self-refinement has been shown to improve the reasoning ability of LLMs within a MAS, but excessive iteration does not guarantee continued improvement. Hence, a further ablation study was conducted to further explore the limitations of both self-refinement and the previously reported ability of MAS learning from past mistakes with episodic memory [27]. The iteration number was doubled for both planning and coding memory with each phase having identical iteration number for simplicity. Figure 2 indicates early findings of an interaction between the number of iterations and the episodic memory configuration. Most noticeably, memory in the planning phase delivers an increase of 10% compared with the 5 iteration version. Whereas, all other results appear to be approximately the same except for a decrease of around 4% when coding memory is used in isolation. This may suggest that planning memory benefits from increased iteration and is able to use all past mistakes constructively. The coding memory might encourage modification without due cause, hence providing a possible explanation for the decrease in success rate and may somewhat explain why memory in both phases may have a cancelling effect.

The average debugging attempts demonstrate that increasing the iteration number also interacts with the placement of episodic memory. The combination of either memory in both stages or none at all appears to increase the debugging attempts required by around one attempt, from 3.3 to 4.3 and from 4.3 to 5.4, respectively. However, the planning and coding memory in isolation show a small decrease and insignificant increase, respectively. Whilst the results do not deliver compelling evidence for trends at higher iteration counts, the large increases

that are visible indicate that without careful infrastructure, more iterations may not provide assurances.

To further understand the debugging attempts and the success rate changes with iteration number, Figure 5 shows the distribution of cyclomatic complexity scores for both iteration counts. The most notable impact is the isolated coding memory violin plot. The distribution appears to be shifted towards a lower complexity compared with its 5 iteration counterpart. Isolated planning memory has a contrasting effect as the distribution appears wider at higher complexity scores compared with the 5 iteration version. A possible explanation may be that coding memory seeks to over-simplify at higher iteration numbers. This may explain the impact on the success rate, whilst planning memory may encourage more detail and criteria for the coding phase to meet.

The LLM-based scoring, shown in Figure 4 shows that for 10 iterations, isolated planning memory produces the highest score at around 59-60%, whereas isolated coding memory offers minimal improvement on the no memory alternative. This data seemingly reflects the trends observed in the success rate and may further support the suggestion that planning memory is reinforced by increased iteration, whilst coding memory may lead to over-simplifications that decrease the quality of the data analysis. Memory in both stages does not seem to be affected by iteration number, suggesting that the benefits of the planning memory may be offset by the observed effects of coding memory.

Overall, these results show that self refinement and the introduction of episodic memory are not guaranteed to improve the system performance without due consideration. Planning memory appears to yield the greatest benefit for the 4B parameter model when 10 iterations are possible. Depending on the use case and the computational cost, the debugging effort required is an acceptable tradeoff for a success rate and LLM-based report quality score that approaches that of a 27B parameter model. Thus indicating that by tolerating an increased latency, a 4B parameter model may be able to narrow the gap influenced by the scaling law [10] but provide a lesser burden on setup cost.

V. Future Work

Whilst the result of the ablation studies are early indicators of the progress towards Edge AI, this is based on limited observations within a single domain. Beyond simply obtaining more observations, future work should aim to assess replication of these results across various scientific domains. Furthermore, task complexity could be assessed to determine whether this has an impact on the evaluation criteria and code successes. A comparison of human evaluation to the LLM-based scoring would improve robustness and accuracy of automated evaluation [29]. Additionally, the development of benchmarks would aid in determining the reasoning ability of MAS for data analysis tasks in light of the present scarcity. Architectural modifications may include global iterations to encourage further exploration of ideas and could use either sequential or parallel workflows [15],[27],[29]. Long-term memory could be implemented over repeat use of the system to enable learning without repeating laborious processes, to reduce latency. This idea may be limited by the context window, thus a specialised RAG system [20] or LLM fine-tuning may be required [40].

VI. Conclusion

This paper studied the potential for an LLM-based MAS to support the progression towards Edge AI and the self-driving lab [9], [41], [4]. Ablation studies were performed using code success rate, average debugging attempts and LLM-based evaluation as a proxy for human judgment [23], [36], [37]. The cyclomatic complexity was used to explain the observed trends with possible links between higher complexity and improved report quality. Episodic memory appeared to offer the most benefit to smaller LLM sizes (4B) while larger LLMs (27B) exhibited consistency across all configurations. At 5 iterations, pre-execution coding memory appeared to show the most improvement at the possible cost of higher observed debugging effort. At higher iterations, coding memory may suffer over-simplification, reducing its effectiveness. By contrast, planning memory appeared to benefit from more iterations, as further exploration may enable continuous improvement [27], whilst potentially requiring a lower debugging effort. Current trends in the latest LLMs show smaller models exhibiting improved reasoning, which may be enhanced by the emergence of MAS design principles. This paper highlights the observed significance of MAS design in narrowing the performance gap influenced by the scaling law [10]. Despite the requirement for further experimentation to deliver conclusive results, these findings provide early evidence that the field is moving towards Edge AI. The potential to reduce the (cost) barrier to entry would enable industry uptake, overcoming the security risks of current cloud based infrastructure [5].

Acknowledgment

I acknowledge the use of GPT-5 to inform the initial research and improve readability, Github Copilot and Cursor for coding assistance, the Hamilton HPC Service of Durham University, Durham University's NCC cluster and CPI's National Biologics Manufacturing Centre for providing the dataset.

References

[1] IBM, "What Is Industry 4.0?" [Online], Available: https://www.ibm.com/think/topics/industry-4-0, Accessed: Jan. 2026.

[2] M. Banner, H. Alosert, C. Spencer, M. Cheeks, S. S. Farid, M. Thomas, and S. Goldrick, "A decade in review: use of data analytics within the biopharmaceutical sector," *Current Opinion in Chemical Engineering*, vol. 34, p. 100758, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2211339821000903

[3] Y.-C. Lin, K.-C. Chen, Z.-Y. Li, T.-H. Wu, T.-H. Wu, K.-Y. Chen, H.-y. Lee, and Y.-N. Chen, "Creativity in LLM-based multi-agent systems: A survey," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 27 584–27 607. [Online]. Available: https://aclanthology.org/2025.emnlp-main.1403/

[4] W. Li, Z. Mao, Z. Xiao, X. Liao, M. Koffas, Y. Chen, H. Ma, and Y. J. Tang, "Large language model for knowledge synthesis and ai-enhanced biomanufacturing," *Trends in biotechnology (Regular ed.)*, vol. 43, no. 8, pp. 1864–1875, 2025.

[5] I. C. Wiest, D. Ferber, J. Zhu, M. van Treeck, S. K. Meyer, R. Juglan, Z. I. Carrero, D. Paech, J. Kleesiek, M. P. Ebert, D. Truhn, and J. N. Kather, "Privacy-preserving large language models for structured medical information retrieval," *NPJ digital medicine*, vol. 7, no. 1, pp. 257–9, 2024.

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[7] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time," *Advances in Neural Information Processing Systems*, vol. 36, pp. 52 342–52 364, 2023.

[8] "How Much is the Nvidia A100 GPU?" [Online], Available: https://northflank.com/blog/nvidia-a100-gpu-cost, accessed: Jan. 2026.

[9] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667345223000196

[10] G. Yin, H. Bai, S. Ma, F. Nan, Y. Sun, Z. Xu, S. Ma, J. Lu, X. Kong, A. Zhang, D. A. Yap, Y. Zhang, K. Ahnert, V. Kamath, M. Berglund, D. Walsh, T. Gindele, J. Wiest, Z. Lai, X. S. Wang, J. Shan, M. Cao, R. Pang, and Z. Wang, "MMAU: A holistic benchmark of agent capabilities across diverse domains," in *Findings of the Association for Computational Linguistics: NAACL 2025*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 4737–4765. [Online]. Available: https://aclanthology.org/2025.findings-naacl.267/

[11] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[12] Google, [Online], Available: https://deepmind.google/models/gemma/gemma-4/, Accessed: Apr. 2026.

[13] J. A. Drozdz and M. R. Ladomery, "The peer review process: Past, present, and future," *British journal of biomedical science*, vol. 81, pp. 12 054–, 2024.

[14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 46 534–46 594.

[15] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, "When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1417–1440, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.78/

[16] W. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943.

[17] A. Turing, "Computing Machinery and Intelligence," *Mind*, vol. 59, pp. 433–460, 1950.

[18] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing Reasoning and Acting in Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[20] R. Infante, *AI Agents and Applications: With LangChain, LangGraph, and MCP*, meap ed. Manning Publications, 2025, version 7.

[21] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, and S. Tang, "Graph Retrieval-Augmented Generation: A Survey," 2024. [Online]. Available: https://arxiv.org/abs/2408.08921

[22] B. Zhao, L. G. Foo, P. Hu, C. Theobalt, H. Rahmani, and J. Liu, "LLM-based Agentic Reasoning Frameworks: A Survey from Methods to Scenarios," 2025. [Online]. Available: https://arxiv.org/abs/2508.17692

[23] R. Wang, J. Guo, C. Gao, G. Fan, C. Y. Chong, and X. Xia, "Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering," *Proceedings of the ACM on software engineering*, vol. 2, no. ISSTA, pp. 1955–1977, 2025.

[24] H. Triem and Y. Ding, ""Tipping the Balance": Human Intervention in Large Language Model Multi-Agent Debate," *Proceedings of the ASIST Annual Meeting*, vol. 61, no. 1, pp. 361–373, 2024.

[25] J. Li, H. Le, Y. Zhou, C. Xiong, S. Savarese, and D. Sahoo, "CodeTree: Agent-guided tree search for code generation with large language models," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 3711–3726. [Online]. Available: https://aclanthology.org/2025.naacl-long.189/

[26] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.

[27] L. Wilson, "Towards self-driving codebases," [Online], Available: https://cursor.com/blog/self-driving-codebases, Accessed: Feb. 2026.

[28] A. Laverick, K. Surrao, I. Zubeldia, B. Bolliet, M. Cranmer, A. Lewis, B. Sherwin, and J. Lesgourgues, "Multi-Agent System for Cosmological Parameter Analysis," 2024. [Online]. Available: https://arxiv.org/abs/2412.00431

[29] C. Lu, R. Lange, Y. Yamada, S. Hu, J. Foerster, D. Ha, and J. Clune, "Towards end-to-end automation of ai research," *Nature*, vol. 651, no. 8107, pp. 914–919, 2026.

[30] C. Gao, X. Lan, N. Li, Y. Yuan, J. Ding, Z. Zhou, F. Xu, and Y. Li, "Large language models empowered agent-based modeling and simulation: a survey and perspectives," *Humanities social sciences communications*, vol. 11, no. 1, pp. 1259–24, 2024.

[31] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, and E. Chen, "A survey on multimodal large language models," *National Science Review*, vol. 11, no. 12, p. nwae403, 11 2024. [Online]. Available: https://doi.org/10.1093/nsr/nwae403

[32] M. Renze, "The effect of sampling temperature on problem solving in large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 7346–7356. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.432/

[33] Openai, "Introducing deep research," [Online], Available: https://openai.com/index/introducing-deep-research/, Accessed: Jan. 2026.

[34] C. Wang, Q. Wu, and the AG2 Community, "Ag2: Open-source agentos for ai agents," 2024, available at https://docs.ag2.ai/. [Online]. Available: https://github.com/ag2ai/ag2

[35] C. Delangue, J. Chaumond, and T. Wolf, "Huggingface," [Online], Available: https://huggingface.co/, Accessed: Jan. 2026.

[36] N. E. Fenton and J. Bieman, *Software metrics : a rigorous and practical approach*, 3rd ed., ser. Chapman Hall/CRC Innovations in Software Engineering and Software Development. Boca Raton, Florida: CRC Press, 2015 - 2015.

[37] Microsoft, "Code metrics - cyclomatic complexity," [Online], Available: https://learn.microsoft.com/en-us/visualstudio/code-quality/code-metrics-cyclomatic-complexity?view=visualstudio, Accessed: Apr. 2026.

[38] Anthropic, "Anthropic," [Online], Available: https://www.anthropic.com/, Accessed: Apr. 2026.

[39] F. Fourati, "A coherence-based measure of agi," 2025. [Online]. Available: https://arxiv.org/abs/2510.20784

[40] X.-K. Wu, M. Chen, W. Li, R. Wang, L. Lu, J. Liu, K. Hwang, Y. Hao, Y. Pan, Q. Meng, K. Huang, L. Hu, M. Guizani, N. Chao, G. Fortino, F. Lin, Y. Tian, D. Niyato, and F.-Y. Wang, "Llm fine-tuning: Concepts, opportunities, and challenges," *Big Data and Cognitive Computing*, vol. 9, no. 4, 2025. [Online]. Available: https://www.mdpi.com/2504-2289/9/4/87

[41] X. Wang, Z. Tang, J. Guo, T. Meng, C. Wang, T. Wang, and W. Jia, "Empowering edge intelligence: A comprehensive survey on on-device ai models," *ACM Comput. Surv.*, vol. 57, no. 9, Apr. 2025. [Online]. Available: https://doi.org/10.1145/3724420